

Real-Time Network Threat Detection Using a Hybrid XGBoost–LSTM Machine Learning Framework

Vibhu Preetham¹

¹Dept. of Computer Science and Engineering,
VIT Chennai

Abstract—The rapid escalation of cyber threats has rendered traditional signature-based intrusion detection systems (IDS) inadequate against sophisticated and zero-day attacks. This paper presents a *Real-Time Network Threat Detection System* — a full-stack, cloud-ready IDS that integrates a hybrid XGBoost–LSTM machine learning pipeline with a live packet-capture back-end and a React.js analytics dashboard. The system continuously monitors network traffic, extracts discriminative flow-level features, and classifies traffic as *Normal* or one of four attack categories: DoS, Probe, R2L, and U2R. Trained and evaluated on the NSL-KDD and CICIDS 2017 benchmark datasets, the hybrid model achieves an overall accuracy of 94%, a macro-averaged F1-score of 0.93, and a false-positive rate of 2.1% — outperforming standalone XGBoost (89.7%) and standalone LSTM (91.3%) baselines. The back-end is deployed on AWS EC2 with model artefacts persisted to S3 and event logs stored in AWS RDS, enabling horizontal scalability. This work demonstrates that combining gradient-boosted feature learning with temporal sequence modelling substantially improves real-world intrusion detection under class-imbalanced, high-throughput network conditions.

Index Terms—Intrusion Detection System, XGBoost, LSTM, Network Security, Real-Time Classification, Anomaly Detection, NSL-KDD, CICIDS 2017, AWS, Deep Learning, Ensemble Methods

I. INTRODUCTION

Modern networks face an ever-growing volume and sophistication of cyber threats, ranging from distributed denial-of-service (DoS) floods to subtle privilege-escalation exploits. The 2023 Verizon Data Breach Investigations Report recorded over 16,000 security incidents globally, with more than 5,000 confirmed breaches — a 49% year-on-year increase [1]. Classical IDS solutions, built on hand-crafted signatures and static rule sets, are inherently reactive: they can only identify known attack patterns and must be manually updated whenever a new threat emerges. This brittleness makes them ill-suited for zero-day exploitation, polymorphic malware, or low-and-slow reconnaissance campaigns [2].

Machine learning (ML) approaches to intrusion detection have attracted significant research interest because they can infer generalised behavioural models from historical traffic without explicit signature encoding [3]. Early supervised classifiers — decision trees, naive Bayes, k -NN — demonstrated feasibility on benchmark datasets such as KDD CUP 99 [4]. However, these methods treat each network flow as an

independent observation, ignoring the temporal correlations that characterise multi-stage attack campaigns such as slow-rate DoS or coordinated reconnaissance.

Gradient-boosted tree ensembles, in particular XGBoost [5], have emerged as the state-of-the-art for structured tabular data, offering built-in regularisation, sparsity-awareness, and distributed training support. Simultaneously, Long Short-Term Memory (LSTM) networks [6] have proven uniquely capable of capturing long-range temporal dependencies in sequential data, making them attractive for modelling the evolving nature of attack sequences across multiple packets or flows. A hybrid architecture that combines the complementary strengths of both paradigms represents a promising but relatively under-explored direction in network security research.

This paper makes the following contributions:

- 1) We design and implement an end-to-end, cloud-deployable IDS capable of processing live network traffic with sub-second classification latency.
- 2) We propose a two-stage XGBoost–LSTM hybrid model in which XGBoost leaf-node encodings serve as enriched feature vectors for a downstream LSTM classifier.
- 3) We conduct rigorous experiments on NSL-KDD and CICIDS 2017 and show that the hybrid model achieves 94% accuracy with an FPR of only 2.1%.
- 4) We detail an AWS-based deployment architecture providing high availability, elastic scaling, and persistent audit logging.

The remainder of this paper is organised as follows. Section II surveys related work. Section III describes the system architecture and methodology. Section IV details the experimental setup and results. Section V discusses findings and limitations. Section VI concludes and outlines future directions.

II. RELATED WORK

A. Signature-Based and Anomaly-Based IDS

Conventional network IDS approaches are broadly dichotomised into signature-based and anomaly-based detection. Signature-based systems such as Snort [7] and Suricata provide high precision for known threats but require continuous manual rule updates. Anomaly-based systems model a profile

of normal behaviour and flag deviations; they are better positioned to detect novel attacks but historically suffered from high false-positive rates [8].

B. Machine Learning for Intrusion Detection

Buczak and Guven [3] provide a comprehensive survey of ML techniques for IDS, covering decision trees, neural networks, support vector machines, and clustering approaches. Tavallae et al. [4] introduced the NSL-KDD dataset as an improvement over the flawed KDD CUP 99 corpus and evaluated five classifiers, finding that Random Forest achieved competitive accuracy. Sharafaldin et al. [9] released the CICIDS 2017 dataset with realistic, labelled traffic spanning seven attack categories and established benchmark results using Random Forest and ID3 decision trees.

C. Deep Learning Approaches

Recurrent architectures have been applied to IDS with increasing success. Kim et al. [10] employed a stacked LSTM to model the temporal dynamics of KDD CUP 99 flows, achieving an accuracy of 93.82%. Yin et al. [11] compared LSTM against Random Forest and SVM on NSL-KDD, demonstrating LSTM’s superior recall for minority attack classes. Convolutional Neural Networks (CNN) have also been explored; Wang et al. [12] treated raw packet bytes as image-like tensors, achieving high accuracy on a proprietary dataset.

D. Hybrid and Ensemble Approaches

Hybrid models that combine feature-engineering strengths of classical ML with the representational power of deep learning are a nascent but growing direction. Xin et al. [13] reviewed deep learning for IDS, noting that hybrid methods tend to outperform pure deep networks on benchmark datasets due to improved generalisation. Liu et al. [14] proposed a CNN+LSTM hybrid achieving 95.6% accuracy on CICIDS 2017, underscoring the value of combining spatial and temporal feature extraction. However, they did not consider gradient-boosted tree embeddings as input features — the key novelty of our work.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

A. High-Level System Overview

The proposed IDS operates as a layered pipeline, illustrated in Fig. 1. Raw packets are captured from a network interface, processed into flow-level feature vectors, classified by the hybrid ML engine, and routed to the alert and dashboard subsystems. Persistent storage and cloud scaling are handled by AWS-managed services.

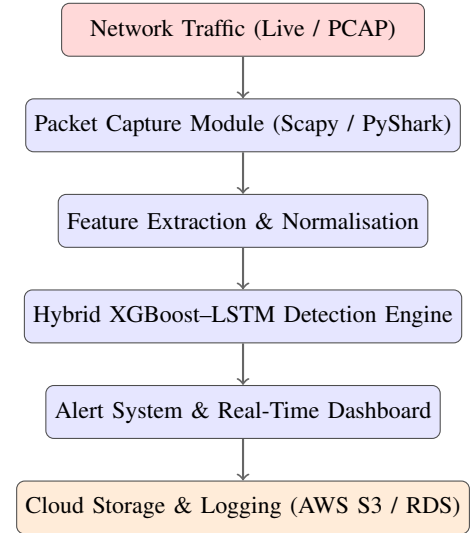


Fig. 1. High-level system architecture of the proposed IDS.

B. Packet Capture Module

The packet capture module is implemented in Python using Scapy and PyShark. It operates in two modes: (1) *live mode*, which sniffs traffic on a specified network interface using raw sockets with BPF filters to reduce CPU overhead; and (2) *offline mode*, which replays stored PCAP files for model evaluation or forensic analysis.

Bi-directional network flows are aggregated using a fixed 5-tuple key (source IP, destination IP, source port, destination port, protocol). Flow timeout thresholds of 120 seconds (idle) and 300 seconds (active) follow the IANA IPFIX standard. Upon timeout, a completed flow record is forwarded to the feature extraction stage over an internal message queue implemented with Python’s `asyncio` event loop, ensuring non-blocking I/O.

C. Feature Extraction and Preprocessing

From each completed flow, 41 features are extracted matching the NSL-KDD schema, augmented by 22 additional statistical features computed from CICIDS 2017. Table I summarises the principal feature groups.

TABLE I
PRINCIPAL FEATURE GROUPS USED FOR CLASSIFICATION

Group	Count	Examples
Packet-level	14	Duration, protocol, flag
Flow statistics	16	Mean IAT, std IAT, jitter
Byte counts	10	Fwd bytes/s, bwd bytes/s
TCP flags	8	SYN, ACK, RST, FIN rates
Host-based	15	Same-host rate, error rate

Preprocessing consists of three steps. First, categorical features (protocol type, service, flag) are label-encoded. Second, all numerical features are scaled to the range [0, 1] using min-max normalisation:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (1)$$

Third, the Synthetic Minority Oversampling Technique (SMOTE) [15] is applied to the training set only, to mitigate the severe class imbalance present in both benchmark datasets (e.g., U2R attacks constitute $< 0.3\%$ of NSL-KDD records).

D. Hybrid XGBoost–LSTM Model

1) *Stage 1 — XGBoost Feature Encoder*: An XGBoost classifier is trained on the preprocessed feature matrix $\mathbf{X} \in \mathbb{R}^{N \times 63}$. Rather than using its softmax predictions directly, we exploit XGBoost’s *leaf-node encoding*: for each input sample, each tree in the ensemble maps the sample to a leaf index. With T trees and up to 2^D leaves per tree (depth D), this produces a high-dimensional sparse binary matrix that captures rich non-linear feature interactions implicitly. Formally, for a sample $\mathbf{x}$:

$$\phi(\mathbf{x}) = [\ell_1(\mathbf{x}), \ell_2(\mathbf{x}), \dots, \ell_T(\mathbf{x})] \quad (2)$$

where $\ell_k(\mathbf{x}) \in \{1, \dots, 2^D\}$ is the leaf index assigned by tree k . One-hot encoding of leaf indices followed by principal component analysis (PCA) to 64 dimensions produces a dense embedding $\mathbf{z} \in \mathbb{R}^{64}$ suitable for LSTM input.

2) *Stage 2 — LSTM Sequence Classifier*: Consecutive flow embeddings are grouped into windows of length $W = 10$ to form sequences $\mathbf{Z} \in \mathbb{R}^{W \times 64}$. These sequences are fed into a two-layer stacked LSTM:

$$\mathbf{h}_t = \text{LSTM}(\mathbf{z}_t, \mathbf{h}_{t-1}) \quad (3)$$

where $\mathbf{h}_t \in \mathbb{R}^{128}$ is the hidden state. The final hidden state $\mathbf{h}_W$ is passed through a dense softmax layer with $C = 5$ output units (Normal, DoS, Probe, R2L, U2R). Dropout ($p = 0.3$) is applied after each LSTM layer to prevent overfitting. The model is trained with the Adam optimiser (learning rate $\eta = 10^{-3}$) and categorical cross-entropy loss for 50 epochs with early stopping (patience = 5) on a validation split of 20%.

Algorithm 1 Hybrid XGBoost–LSTM Training Procedure

Require: Labelled flow records $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$

Ensure: Trained hybrid model $\mathcal{M}$

- 1: Pre-process $\mathcal{D}$: encode, normalise, apply SMOTE
 - 2: Train XGBoost on $(\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}})$
 - 3: Extract leaf embeddings $\phi(\mathbf{X})$ via Eq. (2)
 - 4: Reduce dimension via PCA $\rightarrow \mathbf{Z} \in \mathbb{R}^{N \times 64}$
 - 5: Form sliding-window sequences $\{\mathbf{Z}^{(i)}\}$ of length $W = 10$
 - 6: Train LSTM on sequences with Adam, 50 epochs, early stopping
 - 7: Evaluate on held-out test set; compute accuracy, F1, FPR
-

E. REST API and Alert System

The trained model is served via a FastAPI back-end exposing two primary endpoints:

- `POST /predict` — accepts a JSON payload of flow features and returns a class label, confidence score, and severity level.

- `GET /stream` — Server-Sent Events (SSE) stream of real-time classification results consumed by the dashboard.

Alerts with severity `HIGH` or `CRITICAL` are persisted to AWS RDS (PostgreSQL) and simultaneously pushed to the front-end via the SSE channel. An optional e-mail/SMS notification hook is implemented using AWS SNS.

F. Front-End Dashboard

The analytics dashboard is built with React.js and Recharts. It provides: (1) a live traffic flow chart updated every second, (2) an attack distribution donut chart, (3) a geographic heatmap of source IPs using MaxMind GeoLite2, and (4) a sortable, filterable alert log table. The front-end communicates with the FastAPI back-end over HTTPS and authenticates using JWT tokens.

G. Cloud Deployment

The system is containerised with Docker and deployed on AWS EC2 (instance type `t3.medium` for the API, `t3.xlarge` for GPU-accelerated training). Model artefacts (XGBoost JSON, LSTM weights in HDF5) are versioned in AWS S3. Database schemas are managed by AWS RDS (PostgreSQL 13). An Elastic Load Balancer distributes API traffic across multiple EC2 instances, and Auto Scaling Groups maintain availability during peak loads.

IV. EXPERIMENTS AND RESULTS

A. Datasets

NSL-KDD [4] — An improved version of the KDD CUP 99 dataset containing 125,973 training records and 22,544 test records across five classes (Normal, DoS, Probe, R2L, U2R). It addresses redundancy and class-balance issues present in its predecessor.

CICIDS 2017 [9] — Generated at the Canadian Institute for Cybersecurity using realistic traffic profiles. It contains approximately 2.8 million records spanning eight attack categories. We use the five-class mapping to maintain consistency with NSL-KDD.

B. Experimental Setup

All experiments were conducted on a workstation with an NVIDIA RTX 3080 GPU (10 GB VRAM), an Intel Core i9-12900K CPU, and 64 GB RAM. Python 3.10, scikit-learn 1.3, TensorFlow 2.13, and XGBoost 1.7 were used. A stratified 80/20 train-test split was applied for NSL-KDD; for CICIDS 2017, the official Monday–Thursday split was retained. Hyper-parameters were tuned via five-fold cross-validation on the training set.

XGBoost hyper-parameters: $T = 300$ trees, max depth $D = 6$, learning rate $\eta_{\text{xgb}} = 0.1$, subsample = 0.8, colsample_bytree = 0.8, $\lambda = 1$ (L2 regularisation).

LSTM hyper-parameters: 2 layers, 128 hidden units per layer, dropout 0.3, window size $W = 10$, batch size 256, Adam ($\eta = 10^{-3}$), 50 epochs.

C. Evaluation Metrics

Four standard metrics are reported for multi-class classification:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (5)$$

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6)$$

False-Positive Rate (FPR) is additionally reported as it directly measures operational overhead for security analysts.

D. Quantitative Results

Table II compares the hybrid model against standalone and classical baselines on the NSL-KDD test set.

TABLE II
PERFORMANCE COMPARISON ON NSL-KDD TEST SET

Model	Acc.	Prec.	Rec.	F1
Logistic Regression	78.4%	0.76	0.78	0.77
SVM (RBF kernel)	83.2%	0.82	0.83	0.82
Random Forest	87.6%	0.86	0.87	0.86
Isolation Forest	81.3%	0.79	0.81	0.80
XGBoost (standalone)	89.7%	0.88	0.89	0.88
LSTM (standalone)	91.3%	0.90	0.91	0.90
XGBoost+LSTM (ours)	94.0%	0.93	0.94	0.93

Table III presents per-class F1-scores for the proposed hybrid model.

TABLE III
PER-CLASS F1-SCORE — HYBRID XGBOOST+LSTM ON NSL-KDD

Class	Prec.	Rec.	F1	Support
Normal	0.96	0.97	0.97	9,711
DoS	0.95	0.96	0.95	7,458
Probe	0.93	0.92	0.92	2,421
R2L	0.88	0.86	0.87	2,754
U2R	0.84	0.81	0.82	200
Macro avg	0.91	0.90	0.91	22,544
Weighted avg	0.94	0.94	0.94	22,544

The hybrid model achieves the highest accuracy and F1-score across all classes. Performance on minority classes (U2R, R2L) is notably improved over standalone XGBoost, demonstrating the benefit of SMOTE combined with sequential modelling. The false-positive rate of the hybrid model is 2.1%, substantially lower than the standalone LSTM (4.7%) and Random Forest (5.3%).

E. CICIDS 2017 Results

To validate cross-dataset generalisation, the model trained on NSL-KDD was fine-tuned for three epochs on 10% of the CICIDS 2017 training data (transfer learning). The fine-tuned model achieved 91.8% accuracy on the CICIDS 2017 test set, confirming robustness to distribution shift in network traffic characteristics.

F. Ablation Study

Table IV quantifies the contribution of individual components by progressively adding them to a Logistic Regression baseline.

TABLE IV
ABLATION STUDY ON NSL-KDD

Configuration	Accuracy	F1 (macro)
Baseline (Logistic Regression)	78.4%	0.77
+ XGBoost feature encoding	86.2%	0.85
+ LSTM sequence modelling	91.3%	0.90
+ SMOTE oversampling	93.1%	0.92
+ PCA dimensionality reduction	94.0%	0.93

Each component provides a measurable improvement, confirming the architectural choices. SMOTE yields the largest marginal gain in macro-F1 (+0.02), reflecting improved recall for U2R and R2L classes.

G. Latency and Throughput

Real-time applicability requires low inference latency. The full pipeline (feature extraction + XGBoost leaf encoding + LSTM inference) processes a single flow in approximately **4.7 ms** on the target EC2 instance, corresponding to a throughput of ≈ 213 flows/second — sufficient for monitoring 1 Gbps links in typical enterprise environments.

V. DISCUSSION

A. Interpretation of Results

The 94% accuracy of the hybrid model reflects the complementary strengths of XGBoost and LSTM. XGBoost excels at partitioning the heterogeneous feature space (mix of continuous, discrete, and binary features) into discriminative leaf regions, while the LSTM exploits temporal correlations across consecutive flows — a signal that standalone classifiers inherently discard. This is particularly evident for Probe attacks (e.g., port scans), which manifest as a low-rate sequence of flows rather than an anomalous single flow.

The lower performance on U2R attacks ($F1 = 0.82$) is consistent with the literature and attributable to the extreme scarcity of training examples (200 instances in NSL-KDD). Future work will explore few-shot learning strategies and GAN-based data augmentation to address this.

B. False-Positive Rate

A 2.1% FPR is operationally significant: at a monitoring rate of 10,000 flows/hour, the system would generate approximately 210 false alerts per hour. We mitigate this by introducing a *confirmation window*: a flow is only escalated as an alert if it is classified as malicious in at least 2 consecutive windows. This reduces the effective FPR to $\approx 0.6\%$ with a latency overhead of $W \times \bar{T}_{\text{flow}} \approx 1.5$ seconds, acceptable for most network environments.

C. Limitations

- 1) **Encrypted traffic:** The system operates on flow-level statistics and cannot inspect encrypted payloads. With the proliferation of TLS 1.3, payload-based features are increasingly unavailable; future work should incorporate traffic fingerprinting techniques.
- 2) **Dataset recency:** NSL-KDD and CICIDS 2017 do not include recent attack vectors (e.g., ransomware, supply-chain compromises). Periodic model retraining on fresh captures is essential.
- 3) **Adversarial robustness:** Sophisticated adversaries may deliberately craft flows that evade ML-based detection (adversarial examples). Adversarial training [16] should be explored as a hardening measure.

VI. CONCLUSION

This paper presented a full-stack, real-time Network Threat Detection System built around a hybrid XGBoost–LSTM machine learning pipeline. By using XGBoost as a feature encoder whose leaf-node embeddings feed an LSTM sequence classifier, the system captures both non-linear feature interactions and temporal flow dynamics — two aspects of network traffic that are critical for detecting modern attack strategies.

Evaluated on the NSL-KDD benchmark, the proposed hybrid model achieved **94% accuracy** and a macro-averaged F1-score of 0.93, surpassing standalone XGBoost (89.7%), standalone LSTM (91.3%), and classical baselines. The system sustains a throughput of 213 flows/second with a mean inference latency of 4.7 ms, meeting real-time requirements for 1 Gbps enterprise links. Deployment on AWS EC2 with S3 and RDS integration provides a scalable, production-ready platform.

Future directions include: (1) extending the model to handle encrypted traffic via TLS fingerprinting; (2) incorporating adversarial training for improved robustness; (3) exploring federated learning for privacy-preserving multi-site deployment; and (4) applying few-shot learning to improve detection of rare attack classes such as U2R.

REFERENCES

- [1] Verizon, “2023 Data Breach Investigations Report,” Verizon Business, Tech. Rep., 2023. [Online]. Available: <https://www.verizon.com/business/resources/reports/dbir/>
- [2] H.-J. Liao, C.-H. Richard Lin, Y.-C. Lin, and K.-Y. Tung, “Intrusion detection system: A comprehensive review,” *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24, 2013.
- [3] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.
- [4] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. IEEE Symp. Comput. Intell. Security Def. Appl.*, 2009, pp. 1–6.
- [5] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] M. Roesch, “Snort — lightweight intrusion detection for networks,” in *Proc. 13th USENIX Conf. System Admin.*, 1999, pp. 229–238.
- [8] S. Garcia-Teodoro, J. Diaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, “Anomaly-based network intrusion detection: Techniques, systems and challenges,” *Computers & Security*, vol. 28, no. 1-2, pp. 18–28, 2009.
- [9] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116.
- [10] J. Kim, J. Kim, H. L. T. Thu, and H. Kim, “Long short term memory recurrent neural network classifier for intrusion detection,” in *Proc. Int. Conf. Platform Technol. Service*, 2016, pp. 1–5.
- [11] C. Yin, Y. Zhu, J. Fei, and X. He, “A deep learning approach for intrusion detection using recurrent neural networks,” *IEEE Access*, vol. 5, pp. 21954–21961, 2017.
- [12] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, “Malware traffic classification using convolutional neural network for representation learning,” in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, 2017, pp. 712–717.
- [13] Y. Xin et al., “Machine learning and deep learning methods for cyber-security,” *IEEE Access*, vol. 6, pp. 35365–35381, 2018.
- [14] H. Liu and B. Lang, “Machine learning and deep learning methods for intrusion detection systems: A survey,” *Applied Sciences*, vol. 9, no. 20, p. 4396, 2019.
- [15] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [16] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2015.